Portfolio Phase 2

Internationale Hochschule Duales Studium

Studiengang: Informatik

RideAndPark - Erarbeitungsphase

Lukas Penner

Matrikelnummer: 4234713

Elsternhorst 13, 35415 Pohlheim

Betreuer/in: Armands Strazds

Inhaltsverzeichnis

1. Umsetzung des Projekts 1
2. Fortschritt und Projektorganisation 1
3. Herausforderungen und Probleme 3
4. Zentrale Entscheidungen 3
5. Reflexion 3
6. Zwischenstand und Ausblick 3
7. Fazit 4

1. Umsetzung des Projekts

In der zweiten Projektphase wurde die in Phase 1 definierte Systemarchitektur praktisch umgesetzt. Der Fokus lag dabei auf der Entwicklung des Backends als zentrale Komponente zur Verarbeitung und Bereitstellung von Parkplatzdaten.

Das Backend wurde mit Node.js und Express realisiert und folgt einer klar strukturierten, mehrschichtigen Architektur (Routing, Controller, Service, Transformer). Es implementiert die in Phase 1 geplante Datenpipeline gemäß dem ETL-Prinzip (Extract, Transform, Load) .

Die Hauptaufgaben des Backends umfassen:

- Abruf externer Parkplatzdaten über APIs
- Transformation heterogener Daten in ein einheitliches Modell
- Bereitstellung der Daten über REST-Schnittstellen
- Unterstützung von Geocoding für zielbasierte Suchanfragen

Zentrale API-Endpunkte sind:

```
GET /api/parkings
GET /api/parkings/:id
POST /api/parkings/refresh
GET /api/geocode
```

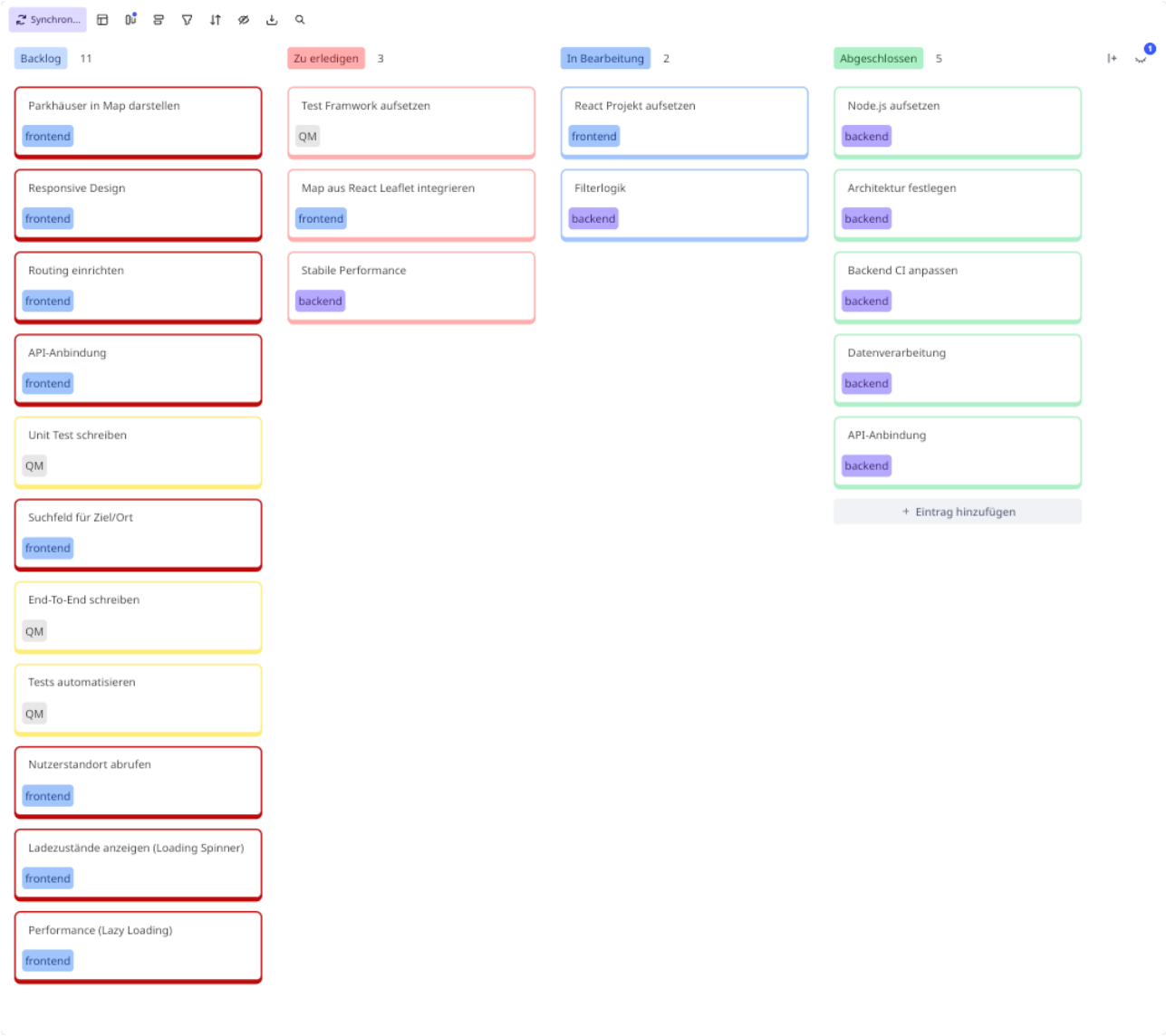
Die interne Verarbeitung erfolgt über einen Transformationslayer, der externe Datenstrukturen vereinheitlicht und in ein konsistentes Domain-Modell überführt.

2. Fortschritt und Projektorganisation

Ein wesentlicher Fortschritt dieser Phase ist die erfolgreiche Implementierung einer durchgängigen Backend-Datenpipeline. Externe Daten können zuverlässig abgerufen, verarbeitet und bereitgestellt werden.

Parallel zur technischen Umsetzung wurde die Entwicklung über ein Kanban-Board strukturiert.

Abbildung 1: Kanban Board



Quelle: Eigene Darstellung aus Miro: https://miro.com/app/board/uXjVGhDVdxM=/

Die Aufgaben wurden in die Zustände Backlog, Zu erledigen, In Bearbeitung und Abgeschlossen unterteilt. Dabei zeigt sich:

- Grundlegende Backend-Komponenten wie Architektur, API-Struktur und Datenverarbeitung wurden abgeschlossen
- Zentrale Features wie API-Anbindung und Filterlogik befinden sich bzw. befanden sich in aktiver Entwicklung
- Erweiterungen wie Performance-Optimierung und Stabilisierung wurden als nächste Schritte identifiziert

Diese strukturierte Vorgehensweise ermöglichte eine transparente Nachverfolgung des Projektfortschritts sowie eine iterative Weiterentwicklung.

3. Herausforderungen und Probleme

Die größte technische Herausforderung liegt aktuell im Bereich der Backend-Performance.

Ursachen hierfür sind:

- Hoher Aufwand bei der Transformation heterogener Daten
- Mehrfache Verarbeitungsschritte pro Anfrage (API-Aufruf, Transformation, Filterung)
- Serverseitige Berechnung von Filtern (z. B. Radius, Verfügbarkeit)

Zusätzlich besteht eine Abhängigkeit von externen APIs, deren Antwortzeiten und Verfügbarkeit direkten Einfluss auf die Systemleistung haben. Zwar wurden bereits Mechanismen wie Timeouts, Retry-Strategien und Fallback-Daten implementiert, diese lösen das Problem jedoch nur teilweise.

4. Zentrale Entscheidungen

Im Verlauf der Umsetzung wurden mehrere grundlegende Architekturentscheidungen getroffen:

- Zentralisierung der Datenlogik im Backend, um Komplexität vom Frontend fernzuhalten
- Einführung eines Transformationslayers, um externe Datenquellen zu vereinheitlichen
- Nutzung eines ersten Cache-Ansatzes, um externe API-Aufrufe zu reduzieren
- Umsetzung einer klaren Schichtenarchitektur, die Wartbarkeit und Erweiterbarkeit verbessert

Diese Entscheidungen folgen direkt den in Phase 1 definierten Prinzipien (API-First, ETL, modulare Architektur) .

5. Reflexion

Die Umsetzung zeigt, dass die gewählte Architektur grundsätzlich geeignet ist, die Anforderungen zu erfüllen. Insbesondere die Trennung zwischen Datenverarbeitung und Datenbereitstellung erweist sich als sinnvoll.

Gleichzeitig wurde deutlich, dass die Verarbeitung von Echtzeitdaten komplexer ist als initial angenommen. Der Transformationsprozess stellt aktuell den größten Engpass dar und beeinflusst sowohl Performance als auch Wartbarkeit.

Ein zentraler Schwachpunkt ist die bislang fehlende systematische Performanzanalyse. Ohne konkrete Messwerte ist eine gezielte Optimierung nur eingeschränkt möglich.

6. Zwischenstand und Ausblick

Das Backend ist funktional implementiert und bildet die Grundlage für die Anwendung.

Aktueller Stand:

- REST-API für Parkdaten
- Integration externer Datenquellen
- Transformationslogik
- Geocoding-Funktionalität
- Fallback-Mechanismen

Offene Schwerpunkte:

- Performanceanalyse und Optimierung
- Erweiterung der Cache-Strategie
- Reduktion der Transformationskomplexität
- Verbesserung der Datenverarbeitungseffizienz

Nächste Schritte:

- Einführung von Laufzeitmessungen zur Identifikation von Bottlenecks
- Optimierung der Transformationslogik
- Erweiterung des Caching-Konzepts
- Verlagerung von Berechnungen auf vorverarbeitete Daten

7. Fazit

Phase 2 markiert den erfolgreichen Übergang der Konzeptions- zur Implementierungsphase. Das Backend ist funktionsfähig und bildet die zentrale Grundlage des Systems. Gleichzeitig zeigt sich klar, dass für eine produktionsreife Anwendung insbesondere die Performance und Stabilität weiter verbessert werden müssen.